

NEURONCLAW

七日 夏令营考核方案

三个 Agent 应用方向与交付要求

A Paper2ARM 科学资产

B 脑病证据溯源与安全解释

C 脑病知识图谱与机制路径

方案定位	在七天内完成一个可运行、可检查、可复现，并能清楚汇报的 Agent 应用原型。
------	---

01 OVERVIEW

一、活动说明

本次 Hackathon 设置三个方向，采用个人参赛形式。每位参赛者选择一个方向，在前六天完成文献调研、方案设计、开发和测试，第七天进行 PPT 汇报、产品演示与答辩。

1. 共同要求

- 提交内容必须能够真实运行，不能只有页面效果图或预先写死的结果。
- 必须展示一个正常案例，以及一个失败、冲突、信息不足或被安全规则阻止的案例。
- 重要结论应能追溯到论文、数据库、指南或输入材料。
- 模型生成内容必须明确标记，不能冒充外部证据。
- 医学相关结果仅用于科研和公共证据整理，不提供自主诊断或处方。

2. 每位参赛者的统一产出

产出	基本要求
可运行原型	一条端到端流程，可通过网页、API 或命令行演示。
文献调研报告	4–6 页，说明相关方法、关键问题以及本人的设计依据。
代码与测试	提供清楚的提交记录、运行命令和基本测试结果。
说明文档	README、流程图、成功案例、失败案例和已知限制。
汇报材料	第七日使用的 PPT 与演示脚本。

3. 文献调研报告要求

- 至少参考 10 个高相关来源，并说明检索平台、关键词、时间范围和筛选方法。
- 用一张对比表概括主要方法、优点、局限和适用场景。

- 提出至少 3 条明确的设计结论，并说明它们如何体现在代码、数据结构或测试中。
- 附真实、准确、可核验的参考文献清单。

关键要求

汇报时需展示至少 3 条“调研发现 → 设计决定 → 实现结果”的对应关系。

A

Paper-to-ARM 科学资产

把一篇或多篇论文整理成可验证、可运行、可回放、可导出的研究资产。

A.1 赛题说明

用户输入一篇或多篇脑科学、神经科学或认知障碍论文。参赛者需要设计一条清晰的处理流程，提取论文中的重要结论、证据和研究方法，并整理为 Agent-Ready Manuscript (ARM) 。

论文输入 → 内容整理 → 结论与证据 → 方法步骤 → 检查 → 试运行与导出

A.2 必须完成

任务	要求
论文输入	支持至少 5 篇真实论文。
ARM 生成	包含基本信息、关键结论、证据、方法步骤、评测计划、来源记录和限制说明。
原文追溯	提取不少于 5 条关键结论，每条都能定位到原文。
试运行与导出	至少一个步骤能够安全试运行，并提供检查、回放和导出结果。
测试	提供运行命令、端到端测试和一个失败案例。

A.3 文献调研重点

- 调研论文解析、图表识别、结论与证据提取、研究方法表达和可复现性。
- 比较不同方法在准确性、可追溯性、可执行性和错误类型方面的差异。
- 回答哪些内容可以自动提取、哪些内容必须人工复核，以及如何避免生成论文中不存在的内容。

通过条件

ARM 内容基本完整；关键结论可回到原文；至少一个步骤可试运行；模型补充内容与论文原文清楚区分。

B

脑病公共证据与安全解释

围绕公开、去标识化案例组织支持证据、反证、信息缺口和风险提示。

B.1 赛题说明

参赛者需要针对一个公开、去标识化的脑病案例，整理相关证据，指出支持信息、反对信息、缺失信息和潜在风险。在信息不足或风险较高时，系统应给出 review_required，而不是直接给出诊断或处方。

公开案例 → 信息整理 → 证据检索 → 支持与反证 → 风险检查 → 安全报告

B.2 必须完成

任务	要求
案例输入	支持至少一个公开、去标识化案例。
证据组织	输出不少于 5 条需要验证的结论，并同时展示支持证据、反证和证据缺口。
来源说明	标明证据来源、发布时间和可靠程度。
安全判断	至少触发一次 review_required，并解释原因。
报告与测试	导出公共证据报告，并提供安全措施辞测试和失败案例。

B.3 文献调研重点

- 调研脑病案例可使用的公开证据来源、证据等级和不确定性表达方式。
- 比较证据质量评估方法，以及系统在信息不足时如何拒答或要求人工复核。
- 回答什么证据可以支持病例级说明、什么时候必须补充信息，以及如何避免过度解读。

安全红线	不诊断、不处方、不使用真实患者隐私；模型推断与外部证据分开；病例报告和试验注册不得被表述为已证实疗效。
------	---

C

脑病知识图谱与机制路径

把论文、ARM 或公开数据库中的知识整理成可追溯图谱，并回答机制问题。

C.1 赛题说明

参赛者需要从论文、ARM 或公开数据库中提取脑病相关知识，完成术语统一、去重、冲突检查和图谱合并，并基于图谱中真实存在的关系回答机制路径问题。

多源输入 → 知识提取 → 统一与去重 → 检查与合并 → 路径查询 → 解释与导出

C.2 必须完成

任务	要求
数据输入	支持论文、ARM 或官方示例数据中的至少一种输入。
增量图谱	生成一个子图，至少覆盖 5 类节点和 6 类关系。
来源与冲突	每条新增关系包含来源和可信度，并展示至少一个冲突或歧义处理案例。
机制路径	支持至少 3 个机制路径问题，路径中的每条关系都真实存在于图中。
展示与测试	提供图谱可视化、导出、基本测试和运行回放。

C.3 文献调研重点

- 调研论文和数据库中的实体、关系如何自动提取、统一和去重。
- 比较知识图谱构建、术语对齐、冲突保留和图谱问答的方法。
- 回答如何避免错误合并、如何区分相关与因果，以及如何确保路径中的每条关系都能核验。

通过条件	新增知识有来源；重复与冲突能够被发现；机制路径来自真实图谱；相关、可能关联和因果关系应清楚区分。
-------------	--

05 SCHEDULE

五、七日安排

前六日完成调研、设计、开发、测试和汇报材料准备。第七日只进行 PPT 汇报、产品演示与答辩，原则上不再新增功能。

日期	重点任务	当天必须完成
Day 1	理解赛题与确定范围	选定输入样例；明确个人目标、主要流程和不做的内容。
Day 2	文献调研与方案设计	完成主要文献筛选；提交流程图、输入输出和 3 条初步设计结论。
Day 3	最小可运行版本	跑通一条最短端到端流程，结果能够真实生成。
Day 4	核心功能完善	完成所选 Track 的主要产出和网页、API 或命令行入口。
Day 5	验证与失败处理	加入检查、风险、冲突、信息不足或失败场景。
Day 6	测试、冻结与汇报准备	完成测试、README、调研报告、PPT 和演示脚本；功能冻结。
Day 7	PPT 汇报与答辩	现场汇报、产品演示和评委提问；不安排常规开发。

第七日 PPT 汇报要求

项目	建议要求
时长	10 分钟汇报与演示，5 分钟答辩。
页数	建议 8–12 页，突出重点，避免大段文字。
必须说明	问题、方案、流程、文献调研结论、演示结果、失败案例、局限和下一步。
必须演示	一条成功路径和一条失败、冲突、信息不足或安全阻断路径。

项目	建议要求
调研转化	展示至少 3 条“调研发现 → 设计决定 → 实现结果”。

六、评分与提交

1. 统一评分

评分项	分值	关注点
端到端完成度	30	能否真实运行，主要产出是否完整。
方案设计	20	流程是否合理，是否充分复用现有平台。
证据、安全与可追溯性	15	来源、风险、冲突和失败处理是否清楚。
文献调研与设计转化	15	调研是否可靠，是否真正影响实现。
代码、测试与复现	10	是否容易运行、检查和继续开发。
PPT 汇报与演示	10	是否清楚、有重点，能否回答关键问题。

2. 各 Track 的评审重点

方向	评审重点
A	ARM 是否完整、可追溯、至少部分可运行，并能检查、回放和导出。
B	是否清楚区分支持证据、反证和缺口，并在风险情况下正确进入 review_required。
C	知识是否有来源、冲突是否保留、机制路径是否真实存在于图谱。

3. 最终提交清单

- 独立 PR 或明确的提交记录，以及可运行原型和运行命令。
- README、流程图、成功案例、失败案例和已知限制。

- 文献调研报告、参考文献清单、测试结果和复现说明。
- 第七日使用的 PPT。

附录（一）：什么是 Agent-Ready Manuscript (ARM) ?

Agent-Ready Manuscript, 简称 ARM, 可以理解为：把一篇传统科研论文整理成一种既方便人阅读，也方便 AI Agent 检查、调用、试运行和复用的结构化科研资产。

它不是新的论文格式，也不是简单把 PDF 转成 Markdown。ARM 的核心，是把论文中的**结论、证据、方法、执行步骤、评测方式和来源记录**拆分成清晰、可关联的对象。

1. 为什么需要 ARM

普通论文主要为人类阅读设计。对于 Agent 来说，论文中很多关键信息分散在：

- * 摘要和正文；
- * 图表及图注；
- * 方法章节；
- * 补充材料；
- * 参考文献；
- * 代码仓库和数据链接。

Agent 即使能够“读懂”论文，也不一定能可靠回答：

- * 论文提出了哪些核心结论？
- * 每条结论由哪段文字、哪个图或哪张表支持？
- * 哪些内容是作者原文，哪些是模型推断？
- * 实验需要什么输入、工具和步骤？
- * 哪些步骤可以直接执行，哪些必须人工确认？
- * 如何判断复现是否成功？
- * 论文及其处理过程能否被追踪和回放？

ARM 就是为解决这些问题设计的。

2. ARM 通常包含什么

一个完整的 ARM 可以包含以下内容：

内容	作用
metadata	论文标题、作者、DOI、版本、许可等基本信息
claims	论文中的关键科学结论
evidence	支持或反驳每条结论的原文、图表和数据位置
protocol	对论文研究方法和实验过程的结构化描述
runbook	Agent 可以理解和执行的操作步骤
eval _ plan	如何验证运行结果是否符合预期
provenance	信息从哪里来、经过什么工具和模型处理
limitations	论文局限、缺失信息和不可自动执行部分
artifacts	代码、配置、运行结果、表格等相关产物

3. “Agent-Ready”具体意味着什么

“Agent-Ready”不代表 Agent 可以无条件自动复现实验，而是至少满足以下条件。

- ① 可理解：论文内容被整理为明确的字段，而不是只有一整篇长文本。
- ② 可追溯：每条结论都能够回到原文段落、图、表或补充材料。
- ③ 可检查：系统可以自动检查：
 - 是否缺少证据；
 - 引用是否完整；
 - 来源和许可是否明确；
 - 是否混入了模型自行补充的内容；
 - 执行步骤是否缺少输入或条件。

- ④ 可执行：论文中的部分方法可以转化为带有输入、输出、工具和失败条件的步骤。
- ⑤ 可回放：能够查看 Agent 是如何解析论文、提取结论、关联证据和生成步骤的。
- ⑥ 可导出和复用：ARM 可以作为一个独立资产包，用于后续研究、知识图谱构建、实验设计或其他 Agent 任务。

Agent 任务。

4. 一个简单例子

假设论文中写道：

> 某种干预在动物模型中降低了海马炎症标志物，并改善了空间记忆表现。

传统论文中，这句话可能分散在摘要、结果、图表和讨论中。

转成 ARM 后，可能整理为：

```
```yaml
```

```
claim:
```

```
 id: C-001
```

```
 text: 该干预在当前动物模型中降低海马炎症并改善空间记忆
```

```
 type: experimental_result
```

```
 status: supported
```

```
```
```

对应证据：

```
```yaml
```

```
evidence:
```

```
 - source: Figure 3
```

```
 location: Results, Section 3.2
```

```
 relation: supports
```

```
 - source: Supplementary Table S2
```

```
relation: supports
```

```
'''
```

对应方法步骤：

```
```yaml
```

```
runbook_step:
```

```
  action: 计算炎症标志物组间差异
```

```
  input: biomarker_table.csv
```

```
  tool: statistical_analysis
```

```
  expected_output: effect_size_and_p_value
```

```
  can_dry_run: true
```

```
'''
```

对应限制：

```
```yaml
```

```
limitations:
```

- 结果来自动物模型，不能直接推广到临床人群
- 原始数据未完全公开

```
'''
```

这样，Agent 不仅知道论文“说了什么”，还知道：

- \* 依据在哪里；
- \* 适用范围是什么；
- \* 哪部分能够验证；
- \* 哪部分不能自动执行。



## 附录（二）Agent workflow 构建

一个实用的 Agent 可以概括为六部分：

模型 + 指令 + 工具 + 运行循环 + 状态 + 安全与追踪

OpenAI Agents SDK 将 Agent 定义为“带有指令和工具的模型”，并提供工具调用循环、会话状态、Guardrails、人工审批和运行追踪。

### 1. 先明确任务与输出

不要先设计多个 Agent。先写清楚：

- 输入是什么；
- 最终输出是什么；
- Agent 可以调用哪些工具；
- 哪些情况必须停止或交给人工；
- 如何判断任务成功。

以脑病证据整理 Agent 为例：

输入：公开、去标识化病例

输出：

- 支持证据
- 反证
- 证据缺口
- 风险标志
- review\_required

建议使用 Pydantic 或 JSON Schema 定义输出，避免最终结果只是一段难以检查的文本。

## 2. 安装 OpenAI Agents SDK

```
python -m venv .venvSOURCE .venv/bin/activate
```

```
pip install openai-agents pydanticexport OPENAI_API_KEY="你的 API Key"
```

官方 Quickstart 使用 `openai-agents` 安装包，并通过 `Agent` 和 `Runner` 创建和运行 Agent。

---

## 3. 创建第一个 Agent

```
from agents import Agent, Runner
```

```
agent = Agent(
```

```
 name="Literature Research Agent",
```

```
 instructions="""
```

```
 你负责分析脑科学文献。
```

```
 工作要求：
```

1. 区分论文原始结论和你的推断。
2. 不得编造文献、实验结果或引用。
3. 输出核心结论、支持证据和研究局限。
4. 信息不足时明确说明。

```
 """ ,
```

```
)
```

```
result = Runner.run_sync(
```

```
 agent,
```

```
 "总结这篇论文的核心结论，并指出证据和局限。",
```

```
)
```

```
print(result.final_output)
```

`Runner` 负责执行 `Agent`，并处理模型调用和后续运行步骤。

## 4. 给 Agent 增加工具

Agent 与普通聊天模型的主要区别，是它可以根据任务调用工具。

```
from agents import Agent, Runner, function_tool

@function_tooldef search_literature(keyword: str) -> str:
 """在本地文献索引中搜索与关键词相关的论文。"""
 # Hackathon 中可以先使用确定性 fixture
 return (
 f"检索结果：与 {keyword} 相关的论文共 3 篇。"
 "包括一篇综述和两篇实验研究。"
)

@function_tooldef validate_reference(reference: str) -> str:
 """检查引用是否包含 DOI、PMID 或其他稳定标识符。"""
 if "doi" in reference.lower() or "pmid" in reference.lower():
 return "reference_valid"
 return "reference_requires_review"

agent = Agent(
 name="Evidence Agent",
 instructions="""
 围绕用户问题组织文献证据。

 在需要文献时调用 search_literature。

 在输出引用前调用 validate_reference。

 不得将模型推断表述为外部证据。

 """,
```

```
tools=[search_literature, validate_reference],
)
result = Runner.run_sync(
 agent,
 "查找轻度认知障碍相关的炎症机制证据。",
)
print(result.final_output)
```

Agents SDK 可以把普通 Python 函数包装为工具，并自动生成参数 Schema；也支持网页搜索、文件搜索、代码执行和 MCP 等工具类型。

## 工具设计原则

每个工具应当：

- 只完成一类明确任务；
- 输入和输出结构清晰；
- 返回真实结果，不返回虚构结果；
- 有超时、异常和空结果处理；
- 高风险操作需要人工审批；
- 保留调用参数和返回结果。

不建议创建一个名为 `do_everything()` 的大型工具。

---

## 5. 增加结构化输出

对于 ARM、证据报告和知识图谱任务，建议使用结构化对象。

```
from pydantic import BaseModel, Field
from agents import Agent, Runner
```

```
class EvidenceReport(BaseModel):
 claim: str
 supporting_evidence: list[str] = Field(default_factory=list)
 counter_evidence: list[str] = Field(default_factory=list)
 evidence_gaps: list[str] = Field(default_factory=list)
 review_required: bool
 review_reason: str

agent = Agent(
 name="Evidence Report Agent",
 instructions="""
 生成结构化证据报告。
 证据不足、来源冲突或存在安全风险时，
 review_required 必须为 true。
 """,
 output_type=EvidenceReport,
)
result = Runner.run_sync(
 agent,
 "现有公开证据是否足以支持该研究结论？",
)
report = result.final_outputprint(report.model_dump())
```

结构化输出便于后续：

- 自动校验；
- 前端展示；
- 写入数据库；
- 构建 ARM；

- 写入知识图谱；
  - 建立自动评测。
- 

## 6. 增加状态和记忆

单轮 Agent 不需要复杂记忆。需要连续对话或长任务时，再加入：

```
thread_id
run_id
messages
claims
evidence
tool_history
open_questions
checkpoint
```

OpenAI Agents SDK 提供 Sessions，用于保存和恢复 Agent 会话状态。

对于 NeuronClaw 这类长流程项目，更适合使用 LangGraph 管理：

- workflow 状态；
- checkpoint；
- 暂停和恢复；
- 人工审批；
- 多阶段 Agent Pipeline。

LangGraph 官方将其定位为面向长时间、带状态 Agent 的低层编排框架，并提供持久化机制。

---

## 7. 增加安全规则

至少实现三类约束：

### 输入规则

- 拒绝真实可识别患者信息；
- 检查文件类型和大小；
- 检查恶意指令或 Prompt Injection。

### 工具规则

- 删除、发布、写数据库等操作需要审批；
- 限制工具参数；
- 只开放当前任务需要的工具。

### 输出规则

- 不伪造引用；
- 不把模型推断当成证据；
- 不输出自主诊断或处方；
- 证据不足时返回 `review_required`。

Agents SDK 的 Guardrails 可以对输入、输出和工具调用执行校验；高风险失败可以提前中止运行。

---

## 8. 增加追踪和评测

每次运行至少记录：

用户输入

Agent 步骤

工具名称

工具参数

工具结果

最终输出

错误与重试

耗时

token 或成本

Agents SDK 内置 tracing，可以查看 Agent 运行、工具调用和多 Agent 交接过程。

建议为 Hackathon 准备 10—20 个固定测试样例，评估：

- 任务是否完成；
- 工具是否选对；
- 输出字段是否完整；
- 引用是否真实；
- 是否正确触发 `review_required`；
- 同一输入是否获得稳定结果。

---

## 多 Agent 什么时候使用

先构建单 Agent。只有角色确实不同，才拆成多个 Agent。

例如 Paper-to-ARM：

Orchestrator

└── 文档解析 Agent

└── Claim 提取 Agent

└── Evidence 对齐 Agent

└── Runbook 生成 Agent



└── Validation Agent

常见组织方式有两种：

### Manager 模式

主 Agent 保持控制权，将子任务分配给专业 Agent。

### Handoff 模式

当前 Agent 将任务直接交给另一个 Agent，由后者继续执行。

OpenAI 官方文档将这两种方式分别称为 agents as tools 和 handoffs。

对于七日 Hackathon，建议：

先完成一个 Agent 加两个工具的可靠闭环，再增加第二个 Agent。

---

## 推荐的最小项目结构

```
agent_project/
├── main.py
├── agents/
│ ├── orchestrator.py
│ └── evidence_agent.py
├── tools/
│ ├── literature_search.py
│ └── reference_validator.py
├── schemas/
│ └── evidence_report.py
├── prompts/
│ └── evidence_agent.md
```

```
|— fixtures/
| |— sample_papers.json
|— tests/
| |— test_agent_pipeline.py
```

---

## 官方文档

### OpenAI

#### 1. OpenAI Agents SDK Overview

Agent、工具、Handoff、Guardrails、Sessions 和 Tracing 总览。

#### 2. OpenAI Agents SDK Quickstart

从安装到第一个 Agent、函数工具和多 Agent 路由。

#### 3. OpenAI Agents SDK — Tools

Function Tools、Hosted Tools、Agents as Tools 和审批机制。

#### 4. OpenAI Agents SDK — Sessions

Agent 会话状态和持久化。

#### 5. OpenAI Agents SDK — Guardrails

输入、输出和工具安全校验。

#### 6. OpenAI Agents SDK — Tracing

Agent 运行追踪、调试和可观测性。

#### 7. OpenAI API — Agents Guide

Agents SDK、Responses API、工具、状态和评测入口。

#### 8. OpenAI API — Using Tools

网页搜索、文件搜索、函数调用和 MCP 工具。

### LangGraph

## 1. LangGraph Overview

适合构建持久化、可恢复、长时间运行的 Agent 工作流。

## 2. LangGraph Persistence

Checkpoint、Thread 和运行状态持久化。

## MCP

### 1. Model Context Protocol Introduction

用标准接口将 Agent 连接到外部工具和数据源。

---

## 面向本项目的推荐架构

LangGraph

负责流程、状态、暂停、恢复和安全门控



专业 Agent

负责规划、工具选择和结果修订



Capability Registry

KG / RAG / Literature / Validation / Safety



结构化输出

ARM / Evidence Report / Incremental KG



Trace、Replay 与 Evaluation

七日考核的最低合格实现应当是：

一个明确任务

+ 一个 Agent

+ 两个真实或 fixture-backed 工具

- + 一种结构化输出
- + 一个失败或 `review_required` 分支
- + 一组自动测试
- + 一份运行记录

## 附录（三）LLM 支持

考核期间提供 DeepSeek V4 的 API 支持（100 元额度，考核结束后 API 关闭）：**sk-**

**03915a5bb5914d6fbb4faed0b0622d2f**

使用方法请参考 DeepSeek API 官方使用教程：<https://api-docs.deepseek.com/zh-cn/>

## 参考文献

- [1] Bran, Andres M., et al. “ChemCrow: Augmenting Large-Language Models with Chemistry Tools.” arXiv, 2023, doi:10.48550/arXiv.2304.05376.
- [2] Gao, Shanghua, et al. “Empowering Biomedical Discovery with AI Agents.” arXiv, 2024, doi:10.48550/arXiv.2404.02831.
- [3] Gottweis, Juraj, et al. “Towards an AI Co-Scientist.” arXiv, 2025, doi:10.48550/arXiv.2502.18864.
- [4] Huang, Kexin, et al. “Biomni: A General-Purpose Biomedical AI Agent.” bioRxiv, 2025.
- [5] King, Ross D., et al. “On the Formalization and Reuse of Scientific Research.” Journal of the Royal Society Interface, 2011, doi:10.1098/rsif.2011.0029.
- [6] King, Ross D., et al. “The Automation of Science.” Science, vol. 324, no. 5923, 2009, pp. 85–89, doi:10.1126/science.1165620.
- [7] Lu, Chris, et al. “The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery.” arXiv, 2024, doi:10.48550/arXiv.2408.06292.
- [8] Roohani, Yusuf, et al. “BioDiscoveryAgent: An AI Agent for Designing Genetic Perturbation Experiments.” arXiv, 2024, doi:10.48550/arXiv.2405.17631.
- [9] Schmidgall, Samuel, et al. “Agent Laboratory: Using LLM Agents as Research Assistants.” arXiv, 2025, doi:10.48550/arXiv.2501.04227.
- [10] Soiland-Reyes, Stian, et al. “Packaging Research Artefacts with RO-Crate.” Data Science, vol. 5, no. 2, 2022, pp. 97–138, doi:10.3233/DS-210053.
- [11] Wilkinson, Mark D., et al. “The FAIR Guiding Principles for Scientific Data Management and Stewardship.” Scientific Data, vol. 3, 2016, article 160018, doi:10.1038/sdata.2016.18.
- [12] Yamada, Yutaro, et al. “The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search.” arXiv, 2025, doi:10.48550/arXiv.2504.08066.

